CORS 기초 개념과 동작 원리

React와 FastAPI 예제로 이해하는 브라우저의 출처 간 요청 규칙

참고: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

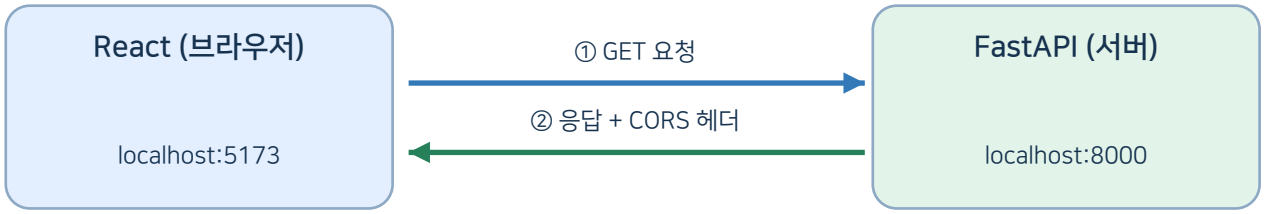
1. CORS란 무엇인가?

CORS(Cross-Origin Resource Sharing)는 서로 다른 출처(Origin)의 웹페이지가 서버 자원을 이용할 수 있도록, 서버가 브라우저에 허용 여부를 알려주는 HTTP 기반 메커니즘입니다.

비유: 브라우저는 보안 담당자, 프론트엔드는 방문객, 백엔드는 사무실, CORS 설정은 방문 허용 명단과 같습니다.

2. CORS가 왜 필요한가?

React가 http://localhost:5173 에서, FastAPI가 http://localhost:8000 에서 실행된다면 포트가 다르므로 두 주소는 서로 다른 출처입니다. 브라우저의 동일 출처 정책(SOP)은 다른 출처의 응답을 JavaScript가 마음대로 읽지 못하게 제한하며, CORS는 서버가 명시적으로 허용한 경우 그 제한을 완화합니다.



③ 브라우저가 허용 출처인지 검사 → 응답 접근 허용/차단

3. 출처(Origin)를 구분하는 기준

출처는 프로토콜 + 호스트 + 포트로 결정합니다. 셋 중 하나라도 다르면 다른 출처입니다.

비교 주소	동일 출처?	이유
http://localhost:5173/users	예	경로만 다름
http://localhost:8000	아니요	포트가 다름
https://localhost:5173	아니요	프로토콜이 다름
http://127.0.0.1:5173	아니요	호스트가 다름

4. CORS의 기본 동작 과정

① React가 FastAPI에 GET 요청 → ② 브라우저가 Origin 헤더를 첨부 → ③ 서버가 CORS 허용 응답 헤더를 반환 → ④ 브라우저가 이를 검사하여 JavaScript의 응답 접근을 허용하거나 차단합니다.

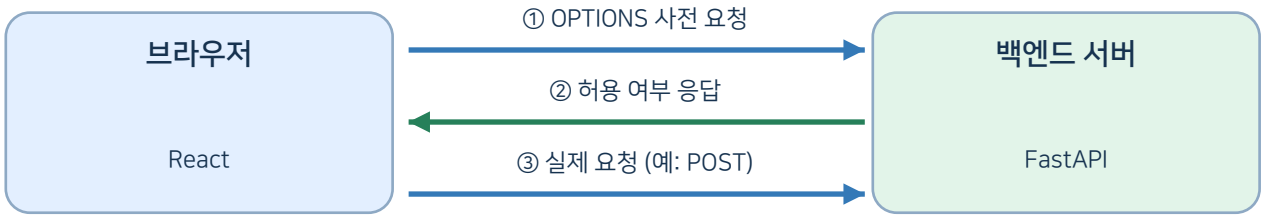
```
Access-Control-Allow-Origin: http://localhost:5173
```

허용 헤더가 없거나 요청 출처와 맞지 않으면 브라우저는 응답 데이터를 읽지 못하도록 차단합니다. 서버가 실제 요청을 처리했더라도 CORS 오류가 나타날 수 있습니다.

5. CORS 요청의 두 가지 대표 흐름

단순 요청(Simple Request): 별도의 사전 요청 없이 실제 요청을 보낸 다음 응답의 CORS 헤더를 검사합니다. 일반적인 GET 요청이 대표적입니다.

사전 요청(Preflight Request): 브라우저가 실제 요청 전에 OPTIONS 요청으로 서버가 메서드 및 헤더를 허용하는지 확인합니다. application/json을 사용한 POST는 일반적으로 여기에 해당합니다.



사전 요청이 통과하면 실제 요청을 보냅니다.
실제 응답에도 적용되는 CORS 검사는 계속 수행됩니다.

주의: 모든 POST 요청에서 사전 요청이 발생하는 것은 아닙니다. 메서드, 헤더, Content-Type 등의 조건에 따라 달라집니다.

6. FastAPI에서 CORS 설정하기

FastAPI의 CORSMiddleware를 이용해 프론트엔드의 출처를 허용할 수 있습니다.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
@app.get("/users")
def get_users():
```

```
    return {"name": "홍길동"}
```

설정 항목	의미
allow_origins	허용할 프론트엔드 출처
allow_credentials	쿠키 등 자격 증명 포함 요청 허용
allow_methods	허용할 HTTP 메서드
allow_headers	허용할 요청 헤더

별표(*)는 광범위한 허용을 의미합니다. 인증 정보가 포함된 요청에서는 와일드카드 출처를 사용하지 말고, 운영 환경에서는 허용 출처·메서드·헤더를 필요한 범위로 제한하는 것이 좋습니다.

7. 반드시 기억할 핵심

CORS 허용 여부는 서버가 HTTP 헤더로 알려주고, 브라우저가 검사합니다.

CORS 오류는 서버 오류와 다릅니다. 서버가 요청을 처리했어도 브라우저에서 응답 접근이 차단될 수 있습니다.

CORS는 로그인·권한 검사를 대신하지 않습니다.

Postman과 Python requests는 일반적인 브라우저 CORS 정책을 강제하지 않습니다.

한 문장 요약: CORS는 서로 다른 출처의 프론트엔드가 백엔드 응답을 읽어도 되는지 서버가 알려주고 브라우저가 판단하는 규칙입니다.